Multidimensional Databases and Data Warehousing

Copyright © 2010 by Morgan & Claypool

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, or transmitted in any form or by any means—electronic, mechanical, photocopy, recording, or any other except for brief quotations in printed reviews, without the prior permission of the publisher.

Multidimensional Databases and Data Warehousing

Christian S. Jensen, Torben Bach Pedersen, and Christian Thomsen

www.morganclaypool.com

ISBN: 9781608455379 paperback
ISBN: 9781608455386 ebook

DOI 10.2200/S00299ED1V01Y201009DTM009

A Publication in the Morgan & Claypool Publishers series
SYNTHESIS LECTURES ON DATA MANAGEMENT

Lecture #9
Series Editor: M. Tamer Özsu, *University of Waterloo*
Series ISSN
Synthesis Lectures on Data Management
Print 2153-5418 Electronic 2153-5426

Multidimensional Databases and Data Warehousing

Christian S. Jensen
Aarhus University, Denmark

Torben Bach Pedersen
Aalborg University, Denmark

Christian Thomsen
Aalborg University, Denmark

SYNTHESIS LECTURES ON DATA MANAGEMENT #9

MORGAN & CLAYPOOL PUBLISHERS

CHAPTER 1

Introduction

This first chapter sets the stage for the remainder of the book. It starts by giving a brief characterization of multidimensional data management and data modeling, then covers briefly the history of multidimensional data management and defines key terms to be used throughout the book. Then it explains why related technologies, specifically, spreadsheets and relations, fall short in meeting the modeling needs of multidimensional data management. Finally, it contrasts multidimensional data management with traditional relational data management.

1.1 MULTIDIMENSIONAL DATA MANAGEMENT

The relational data model, which was introduced by Codd in 1970 and earned him the Turing Award a decade later, constitutes a significant part of the foundation of today's multi-billion-dollar database industry. During the 1990s, a new type of data model, the *multidimensional data model*, emerged that has since made inroads on the relational model when the objective is to *analyze* data, rather than to perform on-line transactions. The multidimensional data model underpins the multi-billion-dollar business intelligence industry, and it plays a role in this industry that is similar to the role that the relational model plays in the database industry.

Multidimensional data models are designed expressly to support data analyses. A number of such models have been proposed by researchers from academia and industry. In academia, formal mathematical models have been proposed, while the industrial proposals have typically been specified more or less implicitly by the concrete software tools that implement them.

Briefly, multidimensional models categorize data as being either *facts* with associated numerical *measures* or as being *dimensions* that characterize the facts and are mostly textual. For example, a book retailer sells *books* at certain *times* in certain *amounts* and at certain *prices*. A typical fact is a *purchase*. Typical measures are the amount and price of the purchase. Typical dimensions represent the location of the purchase, the purchased book, including its genre and author, and the time of the purchase. Queries then aggregate measure values over ranges of dimension values to produce results such as the total sales per month and author.

Multidimensional data models have three important application areas within data analysis. First, multidimensional models are used in *data warehousing*. Briefly, a data warehouse is a large repository of integrated data obtained from several sources in an enterprise for the specific purpose of data analysis. Typically, multidimensional modeling is applied to such data, as this yields good support for data analyses.

Second, multidimensional models lie at the core of *On-Line Analytical Processing* (OLAP) systems. Such systems provide fast answers to queries that aggregate large amounts of so-called detail data to find overall trends, and they present the results in a multidimensional fashion. Consequently, a multidimensional data organization has proven to be particularly well suited for OLAP. The company behind the so-called "BI Verdict" provides an acid test for OLAP by defining OLAP as "Fast Analysis of Shared Multidimensional Information" (FASMI) [70]. In this definition, "Fast" refers to the expectation of response times that are within a few seconds—such response times enable interactive, or on-line, data analysis; "Analysis" refers to the need for easy-to-use support for business logic and statistical analyses; "Shared" suggests a need for security mechanisms and concurrency control for multiple users; "Multidimensional" refers to the expectation that a data model with hierarchical dimensions is used; and "Information" suggests that the system must be able to manage all the required data as well as derived information.

Third, *data mining*, where the aim is to (semi-)automatically discover hitherto unknown knowledge in large databases, is increasingly being applied to multidimensional data. Indeed, it turns out that a multidimensional organization of data is also particularly well suited for the queries posed by data mining tools.

In summary, the multidimensional data model is very powerful for data analysis. It is widely used in industry, and (at least basic) knowledge of it is needed if one wants to analyze and understand large amounts of data.

1.2 MULTIDIMENSIONAL HISTORY

Multidimensional databases do not have their origin in database technology, but they stem from multidimensional matrix algebra, which has been used for (manual) data analyses since the late 19th century.

During the late 1960s, two companies, IRI and Comshare, independently began the development of systems that later turned into multidimensional database systems. The IRI Express tool became very popular in the marketing analysis area in the late 1970s and early 1980s; it later turned into a market-leading OLAP tool and was acquired by Oracle. Concurrently, the Comshare system developed into System W, which was used heavily for financial planning, analysis, and reporting during the 1980s.

In 1991, Arbor was formed with the specific purpose of creating "a multiuser, multidimensional database server," which resulted in the Essbase system. Arbor, now Hyperion, later licensed a basic version of Essbase to IBM for integration into DB2. It was Arbor and Codd that in 1993 coined the term OLAP [13].

Another significant development in the early 1990s was the advent of large *data warehouses* [48], which were typically based on relational *star* or *snowflake* schemas (see Section 2.5), an approach to implementing multidimensional databases using relational database technology.

In 1998, Microsoft first shipped its MS OLAP Server, which was the first multidimensional system aimed at the mass market. This led to the current situation where multidimensional systems

are increasingly becoming commodity products that are shipped at no extra cost together with leading relational database systems.

A more in-depth coverage of the history of multidimensional databases is available in the literature [88].

1.3 RELATED TERMINOLOGY

It is useful to be familiar with a few special terms when studying the literature on multidimensional databases.

OLAP: OLAP abbreviates *On-Line Analytical Processing*. As opposed to the well-known OLTP (On-Line *Transaction* Processing), focus is on data analyses rather than transactions. Furthermore, the analyses occur "On-Line," i.e., fast, "interactive" query response is implied. OLAP systems always employ a multidimensional view of data.

OLAP systems come in three broad categories: systems based on *relational* database management technology, called *ROLAP* systems, systems utilizing non-relational, *multidimensional* array-type technologies, called *MOLAP* systems, and *hybrid* systems that combine these technologies, called *HOLAP* systems.

Data Warehouse: A data warehouse is a repository of integrated enterprise data. A data warehouse is used specifically for decision support, i.e., there is (typically, or ideally) only one data warehouse in an enterprise. A data warehouse typically contains data collected from a large number of sources within, and sometimes also outside, the enterprise.

Data Mart: A data mart is a subset of a data warehouse.

ETL: ETL (Extract-Transform-Load) is the three-step process that feeds a data warehouse. First, an *extraction* process is applied to data from operational source systems, e.g., ERP systems. Second, a *transformation* process is applied in order to change the data format from the source system formats into the data warehouse format. This includes combining data from several different sources and performing *cleansing* to correct errors such as missing or wrong data. Third, a *loading* process is applied to import the data into the data warehouse. ETL is at times also referred to as ETT (Extract-Transform-Transport).

Business Intelligence: Business Intelligence (BI) is the process of making "intelligent" business decisions based on the analysis of available data. From a technology point of view, BI covers the combined areas of data warehousing, reporting, OLAP, data mining, some data visualization, what-if analysis, and special-purpose analytical applications.

1.4 SHORTCOMINGS OF SPREADSHEETS AND RELATIONS

To understand the strengths of multidimensional databases, it is instructive to consider the alternatives. Assume we want to analyze data about sales of books, for which we capture the number of books sold, the book sold, and the city in which it was sold. A simple example with two dimensions is shown in Figure 1.1. When considering how to analyze such data, *spreadsheets* immediately come to mind as a possibility—Figure 1.1 is just a (two-dimensional) spreadsheet.

Book	City			
	Arlington	Boston	Miami	Springfield
Arlington Road Atlas	2072	2	1	5
Gone With the Wind	3984	43765	21923	48959
Italian Cooking	530	9834	4980	7825
Tropical Food	43	1267	15784	248
Winnie the Pooh	863	2345	89	1693

Figure 1.1: Spreadsheet with sales data

Our first analysis requirement is that we do not just want to see sales by book and city combined but also the two kinds of subtotals, sales by book and sales by city, as well as the grand total of sales. This means that formulas for producing the (sub)totals must be added to the spreadsheet, each requiring some consideration. It is possible, if rather cumbersome, to add new data to the spreadsheet, e.g., if new products are sold. Thus, for two dimensions, we can perhaps somehow manage with spreadsheets.

However, if we go to three dimensions, e.g., to include time, we have to consider carefully what to do. The obvious solution is to use separate worksheets to handle the extra dimension, with one worksheet for each dimension value. This will work only when the third dimension has few dimension values, and this will only work to some extent. Analyses involving several values of the third dimension are cumbersome, and with many thousands of, say, time dimension values, the solution becomes infeasible. The situation becomes even worse if we need to support four or more dimensions, which in any case will require a very complex set-up.

Another problem arises if we want to group, e.g., the books, into higher-level categories like "Cooking" and "Fiction." Then we must duplicate the grouping information across all worksheets, which results in a system that uses considerable extra space and is very cumbersome to maintain. The essence of the problem is that spreadsheets tie the data storage too tightly to the data presentation—the *structure* of the data and the *desired views* of the data are not separated. However, spreadsheets are good for *viewing and querying* multidimensional data, e.g., using *pivot tables*.

A pivot table is a 2-dimensional table of data with associated subtotals and totals. For example, if we add subtotals by City and Book and a City/Book grand total to Figure 1.1, we have an example

of a pivot table as shown in Figure 1.2. To support viewing of more complex data, several dimensions may be nested on the x or y axes, and data may be displayed on multiple pages, e.g., one for each book. Pivot tables generally also offer support for interactively selecting subsets of the data and changing the displayed level of detail.

		City			
Book		Arlington	Boston	Miami	Springfield	Total
	Arlington Road Atlas	2072	2	1	5	**2080**
	Gone With the Wind	3984	43765	21923	48959	**118631**
	Italian Cooking	530	9834	4980	7825	**23169**
	Tropical Food	43	1267	15784	248	**17342**
	Winnie the Pooh	863	2345	89	1693	**4990**
	Total	**7492**	**57213**	**42777**	**58730**	**166212**

Figure 1.2: Pivot table with sales data

With spreadsheets falling short in meeting our requirements for the management of multidimensional data, we may then consider using an SQL-based, relational system for data management, as the relational model offers considerable flexibility in the modeling and querying of data. The problem here is that many desirable computations, including cumulative aggregates (sales in year to date), totals and subtotals, and rankings (top 10 selling books), are hard or impossible to formulate in standard SQL.

The main underlying issue is that *interrow* computations are difficult to express in SQL—only *intercolumn* computations are easy to specify. Additionally, transpositions of rows and columns are not easily possible, but rather require the manual specification and combination of multiple views. Although extensions of SQL, such as the *data cube operator* [28] and *query windows* [21], advanced by standards bodies, will remedy some of the problems (see Section 2.8.6 for details), the concept of hierarchical dimensions remains to be handled satisfactorily.

To summarize, neither spreadsheets nor relational databases fully support the requirements posed by advanced data analyses, although it should be noted that these technologies may be adequate under more restricted circumstances. For example, if we have only few dimensions, do not need hierarchical dimensions, and the data volume is small, spreadsheets may provide adequate support. However, the only robust solution to the above problems is to provide data models and database technology that offer inherent support for the full range of multidimensional concepts.

1.5 OLAP VERSUS OLTP

On-Line Analytical Processing (OLAP) is very different from traditional On-Line Transaction Processing (OLTP). When doing OLTP, the focus is on single transactions. Consider, for example, a simple OLTP system used by a book retailer to keep track of the books in stock. When a copy of a

certain book *B* is sold, the system decreases the number showing how many copies of *B* are in stock. Similarly, the number is increased when more copies are delivered from the publisher. The system only keeps track of the most current data, i.e., what is currently in stock. Detailed information about individual sales is either not stored at all or it is not stored in a format amenable to analysis. The retailer may know who the customer is (for example, if the customer pre-ordered the book), but this information is not kept for the purpose of further analysis. The system stores its information in a relational database that is *normalized* to avoid update anomalies. The OLTP system is thus optimized for retrieving and/or updating a small amount of tuples (e.g., "decrease number in stock of book *B* by 1").

In contrast, an OLAP system for the book retailer keeps track of each individual sale. This also includes specific information about the sale such as the city where the sale took place, the date of the sale, the salesperson who sold the book, and information about the book. A multidimensional database is used for the storage of such data. This makes it easy to find, e.g., the most sold cookbook and the average number of children's books sold before a holiday. The OLAP system generally never deletes nor updates its data; only additions of new data take place periodically. The OLAP system is thus optimized for retrieving (and summarizing) very large amounts of data (e.g., "find the average monthly number of sold children's books during the previous three years").

CHAPTER 2

Fundamental Concepts

In this chapter, we cover the fundamental concepts of multidimensional databases. We first cover *cubes*, *dimensions*, *facts*, and *measures*. Then we show how these can be represented in relational databases. After that, we present *data warehouses* and *data marts*. We finally cover data analysis and querying using multidimensional databases.

Throughout the book, we illustrate concepts by considering sales in a fictitious book retailer which has shops in different cities. We keep the examples relatively small, but at the end of this and the following chapter, we summarize important concepts by means of more complete and realistic examples.

2.1 CUBES

A *cube* is a truly multidimensional data structure for capturing and analyzing data. A cube generalizes the tabular spreadsheet such that there can be any number of dimensions (and not only two as in spreadsheets). In addition, hierarchies in dimensions and formulas are first-class, built-in concepts, meaning that these are supported without duplicating their definitions. Although the term "cube" suggests that there are three dimensions, a cube can have any number of dimensions. For that reason, the term *hypercube* is sometimes used instead of cube. A collection of related cubes is commonly referred to as a *multidimensional database* or a *multidimensional data warehouse*.

Figure 2.1 illustrates a three-dimensional cube showing sales of books. We assume that Table 1.1 contains aggregated sales data for year 2008. The cube then adds a time dimension and contains sales counts for two cities, two books, and two years. (A real-world cube would contain counts for many more books, cities, etc., but to maintain readability, we keep the numbers low in this example.)

A cube consists of uniquely identifiable *cells* at each of the dimensions' intersections. A non-empty cell is called a *fact*. The example in Figure 2.1 has a fact for each combination of time, book, and city where at least one sale was made. A fact has a number of *measures* associated with it. These are numerical values that "live" within the cells. In the shown example, we have only one measure, the sales count. A cell may also be empty, meaning that there is no information to record for the given dimension values. An empty cell in the cube in Figure 2.1 thus means that the given book was not sold in the given city in the given year. Depending on the specific application, a highly varying percentage of the cells in a cube is non-empty, meaning that cubes range from being *sparse* to being *dense*. Cubes tend to become increasingly sparse with increasing dimensionality and with increasingly finer granularities of the dimension values.

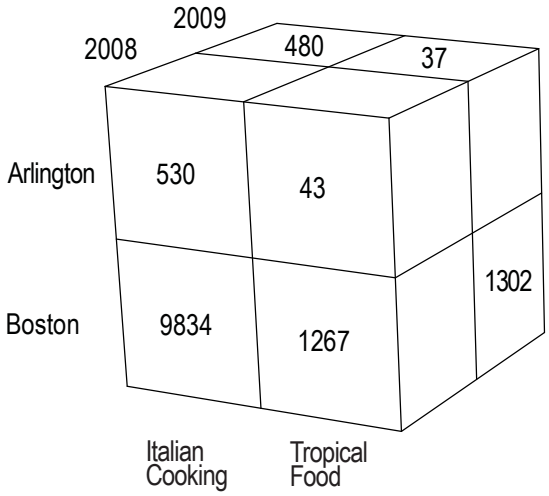


Figure 2.1: Sales data cube

It turns out that most real-world cubes have 4–12 dimensions [48; 88]. Although there is no theoretical limit to the number of dimensions, some tools may experience performance problems when the number of dimensions is greater than 10–15. Generally, only 2 or 3 of the dimensions may be viewed at the same time. However, for low-cardinality dimensions, up to 4 dimensions can be shown by nesting one dimension within another on the axes. Thus, the dimensionality of a cube is reduced at query time by *projecting* it down to 2 or 3 dimensions and *aggregating* of the measure values across the projected-out dimensions. For example, if we want to view sales by City and Time, we aggregate over the entire dimension that characterizes the sales by Book for each combination of City and Time. For the cube in Figure 2.1, we find that the total sales for Arlington in 2008 is 573 by adding up the two numbers 530 and 47.

An important goal of multidimensional modeling is to "provide as much context as possible for the facts" [48]. The concept of *dimension* is the central means of providing this context. One consequence of this is a different view on *data redundancy* than in relational databases. In multidimensional databases, controlled redundancy is generally considered appropriate, as long as it considerably increases the information value of the data. One reason to allow redundancy is that multidimensional databases are often *derived* from other data sources, e.g., data from a transactional relational system, rather than being "born" as multidimensional data, meaning that updates are rare and can be handled more easily [48]. However, there is usually no redundancy in the facts, only in the dimensions.

Having introduced the cube, we cover its principal elements—dimensions, facts, and measures—in more detail.

2.2 DIMENSIONS

The notion of a dimension is an essential and distinguishing concept for multidimensional databases. Dimensions are used for two purposes: the *selection* of data and the *grouping* of data at a desired level of detail.

A dimension is organized into a containment-like hierarchy composed of a number of *levels*, each of which represents a level of detail that is of interest to analyses to be performed. The instances of the dimension are typically called *dimension values* or *dimension members*. Each such value belongs to a particular level.

Figure 2.2 shows the schema and instances of a sample *Location* dimension for the data in Table 1.1.

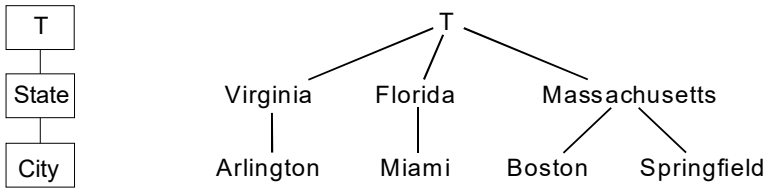


Figure 2.2: Schema and instance for the Location dimension

The Location dimension has three levels, the *City* level being the lowest. *City* level values are grouped into *State* level values, i.e., states. For example, Miami is in Florida. The $\top$ ("top") level represents *all* of the dimension, i.e., every dimension value is part of the $\top$ value.

When using a cube, relevant parts of the data are selected by means of the dimensions. Recall that each cell in a cube is identified by a combination of dimension values. For example, finding the number of sold copies of "Winnie the Pooh" in Boston is just a matter of constraining the Location dimension to "Boston" and the Book dimension to "Winnie the Pooh." It is also possible to group data by means of a dimension. For example, the book retailer could group the data according to the State level in the Location dimension such that the sales counts from a state's individual cities are aggregated into one number for each state. These and other query operations are described in detail in Section 2.8.

In some multidimensional models, a level may have a number of *level properties* associated with it that are used to hold simple, non-hierarchical information. For example, population size of city can be a level property in the lowest level of the Location dimension.

A dimension hierarchy is defined in the metadata of a cube, or the metadata of the multidimensional database, if dimensions can be shared. This means that the problem of duplicate hierarchy definitions as discussed in Section 1.4 is avoided.

Unlike the linear spaces used in matrix algebra, there is typically no ordering or distance metric on the dimension values in multidimensional models. Rather, the only ordering is the containment of lower-level values in higher-level values. However, for some dimensions, e.g., the Time dimension, an

ordering of the dimension values is available and can be used for calculating cumulative information such as "total sales in year to date."

In some cases, it is advantageous for a dimension to have *multiple hierarchies* defined on it. For example, a Time dimension may have hierarchies for both *Fiscal Year* and *Calendar Year*. Multiple hierarchies share one or more common lowest level(s), e.g., Day and Month, and then group these into multiple levels higher up, e.g., Fiscal Quarter and Calendar Quarter to allow for easy reference to several ways of grouping. This is illustrated in Figure 3.13. Most multidimensional models allow multiple hierarchies. We will cover multiple hierarchies in Section 3.3.5.

Most models require their dimension hierarchies to form *balanced trees*. This means that a dimension hierarchy must have uniform height everywhere. For example, this means that if some shops are contained in departments, all shops must be contained in departments, even small ones. Additionally, direct links between dimension values can only go between immediate parent-child levels—they cannot jump two or more levels. For the schema in Figure 2.2, this means that all cities have to belong to states. This is problematic when representing Washington, D.C., which belongs to no state. Finally, each non-top value has precisely one parent. Thus, a product must belong to exactly one product group. In Section 3.3, we discuss the relaxation of these restrictions.

2.3 FACTS

Facts are the objects that represent the *subjects* of the desired analyses, i.e., the interesting "things," or events or processes, that are to be analyzed to better understand their behavior. In our book retailer example, we wish to analyze sales. In most multidimensional data models, the facts are *implicitly* defined by their combination of dimension values. If a non-empty cell exists for a particular combination, a fact exists; otherwise, no fact exists for that combination.

Next, most multidimensional models require that each fact be mapped to precisely one dimension value at the lowest level in each dimension. A fact then has a certain *granularity*, which is determined by the levels from which its dimension values are drawn. For example, the fact granularity in our example cube is "Year by Book by City." Granularities consisting of higher-level or lower-level dimension levels than a given granularity, e.g., "Year by Book Genre by City" or "Day by Book by City" in our example, are said to be *coarser* or *finer* than the given granularity, respectively.

It is commonplace to distinguish between two kinds of facts: *event facts* and *snapshot facts*. Event facts (at least at the finest granularity) typically model *events in the real world*. This means that there is exactly one fact for each unique event in the overall real-world process that is captured. With a fine granularity, there would thus be one fact for each particular sale of a certain book. With a coarser granularity there would be one fact for each day when one or more sales of the book occured. Examples of event facts include sales, clicks on web pages, and flow of goods in and out of (real) warehouses.

Facts usually have associated numerical properties that are called measures. These are the properties that the user wants to analyze, e.g., item count and dollar amount. They are described in the next section.

If event facts do not have any measures, they are often called *factless facts*. However, this is an unfortunate term, and we prefer to rather call them what they really are: *measureless facts*. These facts are used to record that some event took place, but here the event does not have a numerical property that describes it further. This could, for example, happen if a company wants to record that it contacts a certain customer at a given time as part of a given marketing campaign.

A snapshot fact, also called a *state fact*, models the *state* of a given process at a given point in time. Typical examples of snapshot facts include the inventory levels in stores and warehouses, and the number of users using a web site. For snapshot facts, the same object (e.g., a specific can of beans on a shelf) with which the captured real-world process (e.g., inventory management) is concerned may occur in facts at different time points.

It is important to understand when event and snapshot facts come into existence. Event facts are made to represent events, e.g., sales, that have happened in the real world. In principle, these events can happen unpredictably at any time. Snapshot facts, on the other hand, are typically made to represent the states at predefined times, e.g., the inventory on the first day in each month.

Often, the above different types of facts coexist in a given data warehouse, as they support complementary classes of analyses. Indeed, the same base data, e.g., the movement of goods in a (real) warehouse, may often find its way into three cubes of different types: warehouse flow, warehouse inventory, and warehouse flow in year-to-date.

2.4 MEASURES

A *measure* has two components: a *numerical property* of a fact, e.g., the sales price or profit, and a *formula* (most often a simple aggregation function such as SUM) that can be used to combine several measure values into one. In a multidimensional database, measures generally represent the properties of the chosen facts that the users want to study, e.g., with the purpose of optimizing them.

The designer of a multidimensional database thus defines what a numerical measure represents. For a snapshot fact representing sales on a given day, the measure would typically represent the amount of items sold during that day. But the designer could also define the measure to be *cumulative* (i.e., only growing), meaning that the measure represents the total sales on that day and all days before it.

Measures take on different values for different combinations of dimension values. Indeed, a measure can in mathematical terms be considered a partial function from the Cartesian product of the dimensions in the cube to a set of numbers like $\mathbb{N}$ or $\mathbb{R}$. The property and formula are chosen such that the value of a measure is meaningful for all combinations of aggregation levels (including the "top" level $\top$). The formula is defined in the metadata and is thus not replicated as in the spreadsheet example.

Although most multidimensional data models have measures, some do not. In these, dimension values are also used for computations, thus obviating the need for measures, but at the expense of some user-friendliness [68].

It is important to distinguish among three classes of measures, namely *additive*, *semi-additive*, and *non-additive* measures, as these behave quite differently in computations.

Additive measure values can be combined meaningfully along any dimension. For example, it makes sense to add the total sales over Book, Location, and Time, as this causes no overlap among the real-world phenomena that caused the individual values. Additive measures occur for any kind of fact.

Semi-additive measure values cannot be combined along one or more of the dimensions, most often the Time dimension. Semi-additive measures often occur when the fact is of type snapshot. For example, it does not make sense to sum inventory levels across time, as the same inventory item, e.g., a specific book, may be counted several times, but it is meaningful to sum inventory levels across books and stores.

Non-additive measure values cannot be combined along any dimension, usually because of the chosen formula. For example, this occurs when averages for lower-level values cannot be combined into averages for higher-level values. Non-additive measures can occur for any kind of fact.

2.5 RELATIONAL REPRESENTATIONS

After having introduced the fundamental concepts in the previous sections, we are now ready to consider how to represent these in a relational database. Note that other representations of multi-dimensional concepts than the relational also exist. Chapter 4 offers further detail. The relational representations are considered here because they are widely used and are easy to grasp. There are two principal ways of representing dimensions in a relational database; we describe the two in turn.

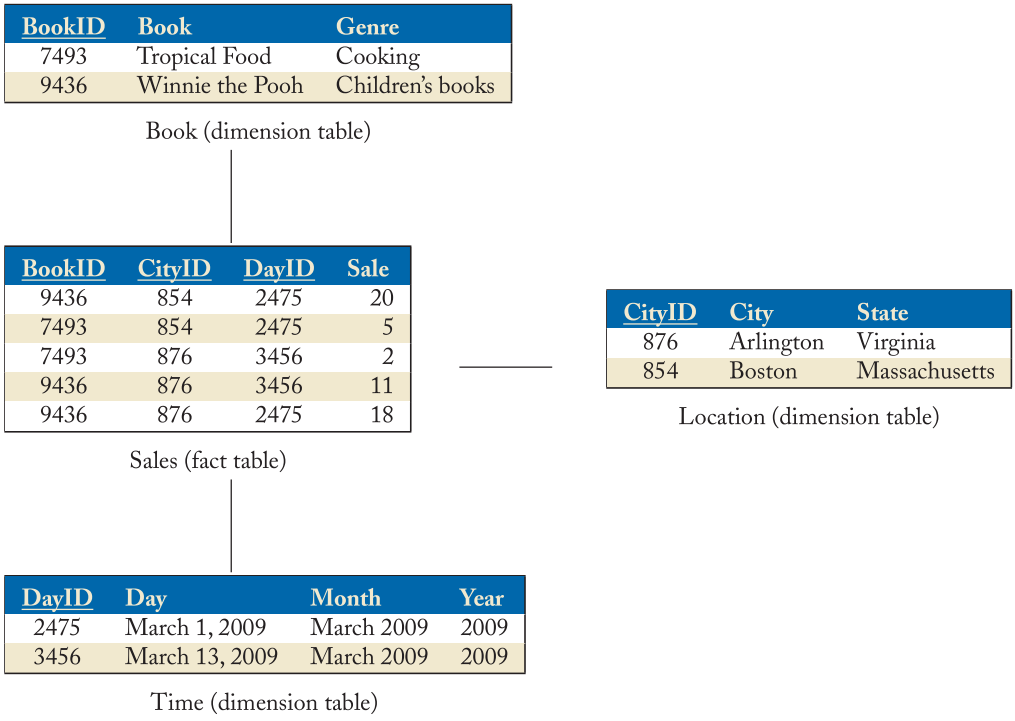
2.5.1 STAR SCHEMAS

A *star schema* has one *dimension table* for each dimension. This table has a key column and one column for each level of the dimension (except $\top$). A column for a level holds textual descriptions of the dimension values at that level. Finally, the dimension table contains one column for each level property in the dimension.

Further, a star schema has a *fact table* that holds a row for each multidimensional fact. The fact table has one column for each measure. In a row, this column contains the measure value for the fact the row represents. A fact table also has one column for each dimension. In a row, these columns contain foreign key values that reference primary key values of dimension tables.

An example star schema instance for the Sales cube is shown in Figure 2.3 (primary keys are underlined). The name "star schema" refers to the observation that if the tables are drawn such that the fact table is at the center and the dimension tables around it, the resulting figure looks (somewhat) like a star. To help visualize this star metaphor, a line is drawn between the fact table and each dimension table in Figure 2.3.

A row in the fact table of the star schema shown in Figure 2.3 holds the sales count for one particular combination of "Book," "City," and "Day." The fact table also has a foreign key column for each of the three dimensions, Book, Location, and Time. The dimension tables have corresponding



BookID	Book	Genre
7493	Tropical Food	Cooking
9436	Winnie the Pooh	Children's books

Book (dimension table)

BookID	CityID	DayID	Sale
9436	854	2475	20
7493	854	2475	5
7493	876	3456	2
9436	876	3456	11
9436	876	2475	18

Sales (fact table)

CityID	City	State
876	Arlington	Virginia
854	Boston	Massachusetts

Location (dimension table)

DayID	Day	Month	Year
2475	March 1, 2009	March 2009	2009
3456	March 13, 2009	March 2009	2009

Time (dimension table)

Figure 2.3: Star schema for Sales cube

key columns, e.g., "CityID," and one column for each of their levels, e.g., "City," and "State" for the table representing the Location dimension. No column is needed for the $\top$ level, as that column would always hold the same value. The key column in a dimension table is typically a "dumb" integer key without any semantics, i.e., a *surrogate key*. This has several advantages over the option of using information-bearing keys from the source systems, including better storage use, prevention of problems associated with key re-use, better support for dimension updates [48], and more efficient query processing.

It can be seen that there is redundancy in higher-level data. For example, "March 2009" will be present for each day during that month, meaning that the Month column takes the value "March 2009" in 31 rows (assuming that at least one book is sold during each day of that month such that all its days must be represented). However, as dimensions typically take up only 1–5% of the total storage required for a star schema, redundancy is not a problem space-wise; it does not cause update-performance problems. In addition, the updates of dimensions are handled centrally, so it is also possible to ensure consistency. Thus, it is often a good idea to use such redundant dimension tables because these support simpler formulation of (and better-performing) queries than do their normalized counterparts.

2.5.2 SNOWFLAKE SCHEMAS

A *snowflake schema* has a fact table, as does a star schema. Snowflake schemas, however, contain several dimension tables for each dimension, namely one table for each (non-$\top$) level. This means that redundancy is avoided, which may be advantageous in some situations, and it renders the hierarchies in dimensions explicit. The dimension tables contain a key, a column holding textual descriptions of the level values, and possibly columns for level properties. Tables for lower levels also contain a foreign key to the containing level. If the tables are drawn with the fact table in the middle and related dimension tables next to each other, the figure looks similar to a snowflake. Thus the name "snowflake schema."

Figure 2.4 shows an instance of a snowflake schema that holds the same data as the star schema in Figure 2.3. For example, the Day table in Figure 2.4 contains an integer key, a date, and a foreign key to the Month table. Note that with this schema, month values will not be replicated. It is, however, harder to query the schema since several joins must be applied. The many joins also make it more time-consuming for the database management system (DBMS) to compute the result of the query.

The choice of whether to use a star schema or a snowflake schema depends highly on the desired properties of the system being developed. Indeed, it is possible to apply snowflaking only partially to a star schema, either fully to only selected dimensions or partially within a dimension. The term *starflake schema* has been applied to the resulting kind of schema. For brevity, we omit a full discussion of this aspect.

2.6 DATA WAREHOUSES AND DATA MARTS

We previously called a collection of related cubes a *data warehouse*. However, there is more than that to say about data warehouses. Bill Inmon, one of the pioneers of data warehousing, defines a data warehouse as a *subject oriented, integrated, time variant, non-volatile collection of data in support of management's decision making process* [35]. In the following, we consider the implications of the different elements of this definition, this way presenting important perspectives on the concept of a data warehouse.

Starting from the back, it can be seen that the purpose of a data warehouse is to support decision making. This is very different from the purpose of an organization's *operational* systems that support the *daily* business processes (handling of orders, deliveries, bills, etc.) but not strategic analysis and decision making. This different focus impacts both *what* data to store in the data warehouse and *how* to store it.

In Inmon's definition, a data warehouse is *subject oriented*. In other words, a data warehouse is designed around the important subjects that concern the business, to allow easy analysis of them. For a book retailer, this includes "sales." A book retailer's data warehouse is thus designed around sales; and to make analysis of them easy, the sales are described by means of entities such as "books" and "shops." In contrast, operational databases are modeled to support daily operations and are heavily influenced by how the daily processes are carried out (i.e., the workflows) in the business and how

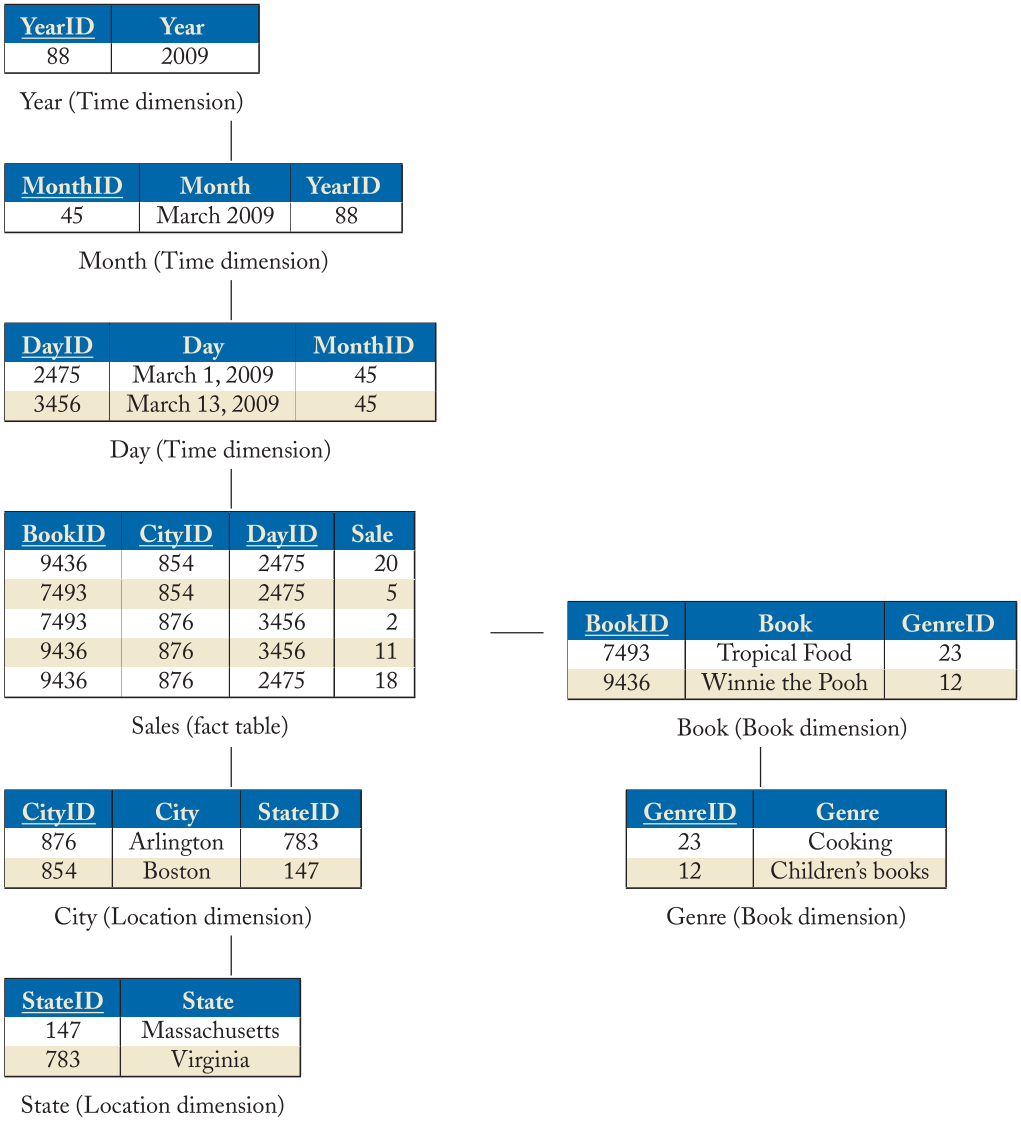


Figure 2.4: Snowflake schema for Sales cube

the operational applications that use them work. The book retailer could, for example, have different operational applications for ordering of books from publishers, distribution to shops, marketing campaigns, and accounting. Analysis of the profit from sales during a campaign then requires the analyst to get data from several different systems that organize their data in different ways and are likely to provide inconsistent data.

Inmon also states that a data warehouse is *integrated*. The data found in a data warehouse will often originate from several different operational systems, e.g., different publishers' book catalogs, a stock keeping system, and the cash registers in shops. Each of these systems may use different formats. For example, one could use the code "P" for paperback, while another system may use "paperback," and a third system may use the numerical code "3." In the data warehouse, a unique code or description is used such that the user does not get confused by different codes that mean the same, or by similar codes with different meanings. We also note that it is considered very bad practice to use cryptic codes in a data warehouse [48]. Instead, understandable, non-abbreviated, textual descriptions should be used (i.e., use "paperback" instead of the confusing "P" or "3"). The different systems might also use different units: for example, some publisher's book catalog may hold the dimensions of books in centimeters while another uses inches. In the data warehouse, only one unit is used. It is the job of the so-called Extract–Transform–Load (ETL) process to extract data from different source systems, clean the data, and transform the data into an integrated format before loading the data into the data warehouse. It is a major part of any data warehouse project to create a working ETL process. We say more about the ETL process in Chapter 4.

A data warehouse is also *time variant*. This means that the data warehouse shows the evolution over time and not just the most recent data, as the operational systems tend to do. In other words, it is possible to see what the modeled world looked like at a certain point in time. For the book retailer, it is then possible to see both how many copies of "Winnie the Pooh" have been sold in total up to now and how many copies were sold by the same time last year. The data warehouse also captures changes in the modeled world, e.g., when one of the book retailer's shops is expanded. It is then possible for the analyst to see the size of the shop at different points in time and investigate how the expansion has affected the sales. We say more about this type of change in the next chapter. We note that virtually any data warehouse has a time dimension.

Finally, Inmon states that a data warehouse is *non volatile*, meaning that neither deletions nor updates are applied to data already in the data warehouse; the only changes are due to the loading of new data. In contrast, the operational systems are designed to support (concurrent) updates and therefore must use advanced transaction handling and normalized designs to enable efficient updates and avoid update anomalies. In a data warehouse, normalization is not needed (recall the star schema from Section 2.5.1). Further, the operational systems typically only include fairly recent data and do not support analysis of historical data.

A *data mart* is generally considered to be a subset of a data warehouse. While a data warehouse is shared throughout an entire organization and can hold data about different subjects, a data mart holds data about a single subject, e.g., sales. Often, a data mart is thus specialized towards the needs of a (sub-)department. So an organization's data warehouse can have cubes and dimensions for both shop sales, Internet sales, stock keeping, and staffing, while a data mart can be built to only hold data about sales, which is the only process the sales department is interested in. Likewise, another data mart can be built to only hold data about staffing, which is what interests the human resource department. Note, however, that the departments should not have their own, private data marts. If

two or more departments need to use the sales data, they should both use the same sales data mart. The data warehouse is loaded with data from different operational source systems, while a data mart is loaded with (already cleaned and integrated) data from the data warehouse or even just provides a logical view of data in the data warehouse.

Unfortunately, there is some disagreement in the literature about what the relationship between data warehouses and data marts should be. Some individuals advocate the point of view that a data warehouse consists of a collection of data marts. This view represents a bottom-up approach in which a data warehouse is built by first building the smaller data marts and then merging these to obtain the data warehouse. This is basically the opposite of the definition given above, where the data marts are derived *from* the data warehouse in a top-down manner. Inmon advocates strongly the top-down approach [36]. Among other problems with the bottom-up approach, he points out its lack of the important integration.

An important problem with the top-down approach is the difficulty of the development process—it can be very challenging to build a data warehouse in a top-down manner for a large organization. As a result, the process may be lengthy, with the return on the investment coming late.

In what we may view as an attempt at obtaining the best of both worlds, Kimball, a pioneer of dimensional data warehousing, introduces the concept of so-called *conformed* dimensions and facts. Here, the organization standardizes the facts and dimensions that are to be shared in the organization. This enables a process with both top-down and bottom-up elements, and it renders it possible to combine several marts or cubes in a coherent fashion. The next section covers the modeling process in more detail.

2.7 MULTIDIMENSIONAL MODELING PROCESSES

A key difference of multidimensional modeling from "ordinary" data modeling is that the multidimensional modeler should neither try to include all the available data nor all the existing relationships in the data in the model. Only those aspects that are essential "drivers" of the business should be included.

Another difference is that redundancy is acceptable in a few, well-chosen places (mostly dimensions) if it makes the model more intuitive to the user. For example, we have seen that a month name is repeated in a time dimension for each day of the month. Information about a customer may also be present in both a person-oriented Customer dimension and a group-oriented Demographics dimension.

Kimball [48; 50] organizes the multidimensional modeling process into four subprocesses.

1. Choose the business process(es) to model.
2. Choose the granularity of the business process.
3. Design the dimensions.
4. Choose the measures.

Step 1 relates to the observation that not all business processes are equally important for the business. For example, for a book retailer, there are business processes for *sales* of books to customers and *purchases* of books from publishers, but the sales process is probably the one with the largest potential for increasing the profits, for which reason this business process should thus be prioritized.

Step 2 implies that it is important to use the data granularity that best matches the analysis needs. For example, "individual sales of books" may be captured, or perhaps (the aggregated) "total sales per book per store per day" may be detailed enough, while enabling performance and storage gains. The grain thus decides what a fact means.

Step 3 then goes on to refine the schema of each part of the grain into a complete dimension with levels and attributes. For the example above, a Store, a Book, and a Time dimension are specified.

Finally, the numerical measures to capture for each combination of dimension values, for example dollar sales, unit sales, dollar cost and profit, are chosen in Step 4.

When performing multidimensional modeling "in the large," with many cubes and several user groups, it is very important to ensure compatibility between cubes such that analysis results are comparable. This is ensured by using so-called *conformed* dimensions and measures [48; 50]. Two dimensions are conformed if they are identical or if one is a subset, with respect to dimension values as well as attributes, of the other. Two measures are conformed if they are equivalently defined.

When conformed dimensions and facts are used, it is possible to meaningfully combine data from different cubes. Instead of re-defining the same concept such as "customer" each time it is used, it is far better to have a single definition that is used in all cubes that have a Customer dimension. Likewise, a measure such as "dollar sales" should be defined in the same way (e.g., include sales tax and not be rounded off) such that it, e.g., is possible to compare the planned dollar sales from the Budget cube with the actual dollar sales from the Sales cube.

The creation of conformed dimensions and measures sounds easier than it is, since it often involves interactions, and possibly mediation and negotiations, with different parts of an organization to define, for example, common Product or Customer dimensions that are acceptable to everyone. Large organizations may have 20+ source systems that are used by different departments that all define the concept "customer" in their own way. To obtain a single definition thus requires strong sponsorship and commitment from the management.

2.8 ANALYSIS AND QUERYING

The objective of a multidimensional database is to support data analyses of large amounts of data. Here, we describe the most important operations available to the analyst.

2.8.1 ROLL-UP, DRILL-DOWN, AND DRILL-OUT

Consider the cube shown in Figure 2.5. This cube has the three dimensions Book, Location and Time. Recall that a cube has a cell for each combination of dimension values. Figure 2.5 shows the cells at the intersections of dimension values from the lowest levels in the hierarchies. Note that for practical reasons, we only show very few values in each dimension. A real-world cube would

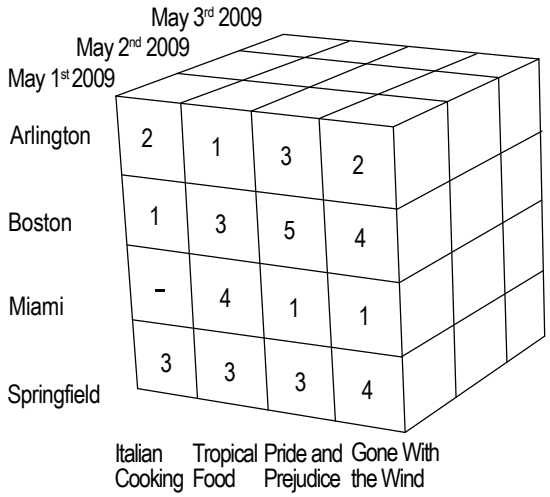


Figure 2.5: A cube

contain many more dimension values. The analyst would, however, rarely or never consider all the data at the lowest levels in the hierarchies. An analyst may want a less detailed view, for example showing the sales per genre instead of the sales per each individual book. To do this, the analyst can *roll-up* to the Genre level in the Book dimension. By doing that, the analyst considers the cells at the intersections of dimension values from each of the lowest levels of Location and Time and dimension values from Book's Genre level. This is shown in Figure 2.6. The cells in this figure holds the number of sales per genre. Each such number is the sum of sales per book in the genre. Recall that a measure consists of a numerical property and a formula—in this case SUM—that is capable of combining, or aggregating, multiple measure values into one. The number of cookbooks sold in Arlington is given by 2 + 1 = 3 as two copies of "Italian Cooking" and one copy of "Tropical Food" were sold there. To roll-up thus means to go from some level in a hierarchy to a higher level in the same hierarchy in the cube considered. In other words, less detail is seen after a roll-up.

It is also possible to roll-up to the Book dimension's $\top$ level. The $\top$ level, by definition, has only one value. The analyst then considers the cells at the intersections of dimension values from Time and Location and the single dimension value from Book's $\top$ level. This corresponds to the analyst considering the two-dimensional cube (or table) showing the sales per combination of Location and Time values such that the Book dimension is projected out. This is shown in Figure 2.7.

The analyst can also roll-up along any of the other dimensions. If a roll-up is done to the $\top$ level for each dimension, the result is a single cell holding the total number of all books sold.

The opposite operation of the roll-up is called a *drill-down*. When doing a drill-down, more details are shown. It is, for example, possible to drill-down from Book's Genre level to its lowest level. This corresponds to going from Figure 2.6 to Figure 2.5. An analyst typically starts out by

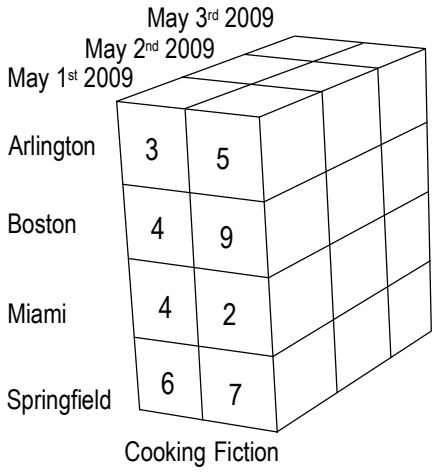


Figure 2.6: The cube from Figure 2.5 rolled-up to Book's Genre level

considering higher-level data and then performs drill-downs to understand the details better. For example, the analyst may consider sales by genre by year and discover that in 2005, many more children's books were sold than in 2004. By doing a drill-down along the Time dimension, the analyst can then discover that the sales were particularly high during July 2005, and by doing a drill-down along the Book dimension, the analyst can see that the increased sales were due to many copies of the, at that time, new book "Harry Potter and the Half-Blood Prince" being sold.

Drilling down offers more detail by "expanding" one or more dimensions from an a non-$\top$ level to a lower level. It is also possible to obtain more detail by including an additional dimension (at a non-$\top$ level)—we refer to that as a *drill-out*. After a drill-out, the measure values are "spread out" among more cells. Assume, for example, that the analyst is considering the number of books sold in each genre such that only the Book dimension is used in the analysis, as shown in Figure 2.8. The analyst can then drill-out using the Time dimension to see the sales by genre by year. This is shown in Figure 2.9.

2.8.2 SLICING AND DICING

Think of a cube as an onion. When an onion is being prepared for cooking, it may be sliced and even diced into small pieces. In context of a cube, the process referred to as slicing and dicing has a similar effect. When analyzing a cube, the analyst may wish to only consider a subset of the cube. To do this, the analyst can select a specific value for a dimension (e.g., Year 2009 from the Time dimension). This way, a *slice* of the cube is obtained as illustrated in Figure 2.10. It is possible to slice the resulting slice further such that the analyst considers a *dice*. A dice can also be obtained by giving more complex conditions to select a subset of the data (e.g., consider the years 2008 and 2009 and books written by Jane Austen). We call the process of creating these subsets *slicing and dicing*.

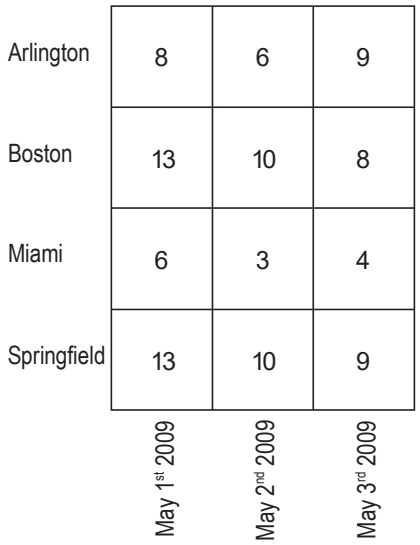


Figure 2.7: The cube from Figure 2.6 rolled-up to Book's ⊤ level

2.8.3 DRILL-ACROSS

So far, we have only considered operations on a single cube. Now assume that our book retailer starts an Internet shop to supplement the physical shops. The book retailer does not want to use the existing Sales cube to capture sales from both physical shops and the Internet shop as the Location dimension describes the locations of physical shops in the U.S. The Internet shop may, however, ship to identifiable customers in different parts of the World. The book retailer instead extends the multidimensional database with an Internet Sales cube. Internet Sales and the old Sales cube share the conformed dimensions Book and Time. In addition, Internet Sales uses a Customer dimension.

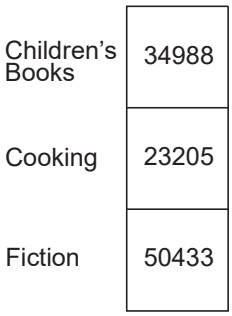


Figure 2.8: Sales by Genre

	2008	2009
Children's Books	16904	18084
Cooking	12101	11104
Fiction	26200	24233

Figure 2.9: Sales by Genre by Year

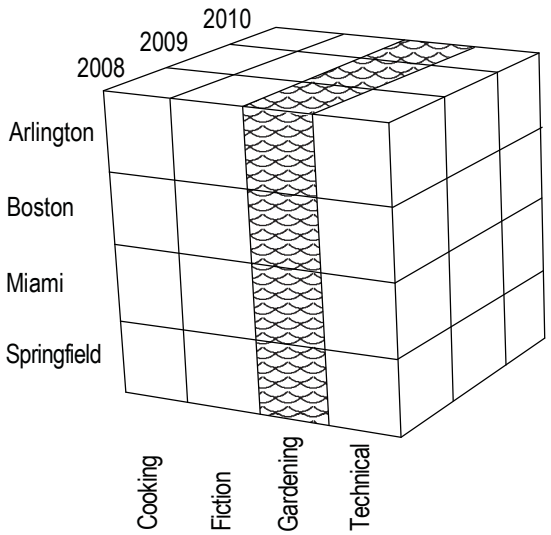


Figure 2.10: A slice

When an analyst considers the sales of books, there are now two cubes to use. However, the analyst wants to see sales figures for the two books that sell the best in total, i.e., considering both shop and Internet sales. To find the best selling books, it is not enough to use the Sales cube to find the books that sell the best in shops and then use the Internet Sales cube to find the books that sell the best on the Internet. To see this, assume that in the physical shops, "Book A" was sold 10,000 times, "Book B" 9,000 times, and "Book C" 8,000 times. From the Internet shop, "Book D" was sold 5,000 times, "Book E" 5,000 times, and "Book C" 4,000 times. If the analyst just found the, say, two best selling books from the cube, the analyst would wrongly conclude that "Book A" sells the best.

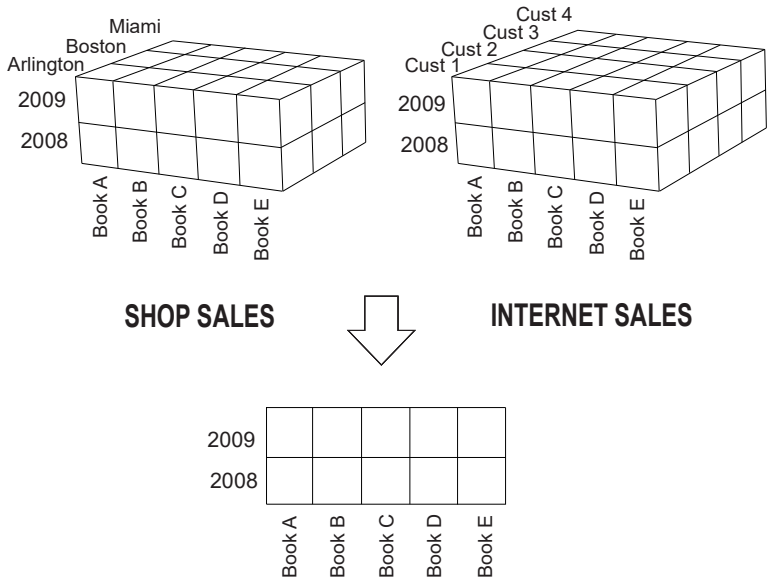


Figure 2.11: A drill-across

To make it possible to see that "Book C" in fact sells the best, the data in the two cubes must be combined.

Data in two cubes can be combined by means of a *drill-across* operation that combines data in two cubes by means of one or more shared dimensions. In relational terms, a drill-across corresponds to a full outer join. Figure 2.11 illustrates the drill-across of the cubes Shop Sales and Internet Sales (for readability, the measure values are not shown).

The resulting cube has as dimensions those dimensions that are shared between the original cubes. Non-shared dimensions can be thought of as rolled up to their top levels in the original cubes before the cubes are combined. The resulting cube inherits all the measures of both the original cubes. These measures are applied to the data in the original cubes. In our example, the analyst should thus consider a *derived measure*, namely the sum of the two cubes' sales measures, to find the best selling books.

When doing a drill-across, the shared dimensions must be conformed, but they need not be identical (remember that two dimensions are conformed if they are identical or if one is a subset of the other). Thus, the cubes may have different granularity. The dimensions in the resulting cube will then have the levels that the original cubes share. For example, the Budget cube can be by genre by year while the Sales cube has finer granularities for the Book and Time dimensions. The data in the Sales cube can then be rolled-up to the Year and Genre levels before the cubes are combined in a drill-across. The measures, showing planned sales and actual sales, respectively, can then be compared meaningfully.

2.8.4 PIVOT TABLES

In Section 1.4, we said that a pivot table is a 2-dimensional table of data with associated subtotals and totals. As an example, consider the pivot table in Figure 2.12 that shows book genres on the y axis and states on the x axis. Note how subtotals and a grand total are also shown.

	Florida	Massachusetts	Virginia	Total
Children's books	2042	8096	4031	**14169**
Cooking	20764	19174	573	**40511**
Fiction	50145	68944	12493	**131582**
Total	**72951**	**96214**	**17097**	**186262**

Figure 2.12: A pivot table

When using a pivot table, it is typically possible to select the data shown on an axis. For example, the user can pick the genres Cooking and Fiction and the states Florida, Massachusetts, and Virginia and then only sees the relevant subset (rows in this case) of the data and the subtotals and totals of the subset.

A pivot table may also allow the user to use hierarchies to drill-down or roll-up. For example, the analyst can "expand" the Cooking row in the pivot table in Figure 2.12 to see the data for the individual books in this genre. This is shown in Figure 2.13.

	Florida	Massachusetts	Virginia	Total
Children's books	2042	8096	4031	**14169**
Cooking	20764	19174	573	**40511**
Italian Cooking	*4980*	*17659*	*530*	*23169*
Tropical Food	*15784*	*1515*	*43*	*17342*
Fiction	50145	68944	12493	**131582**
Total	**72951**	**96214**	**17097**	**186262**

Figure 2.13: The pivot table with the Cooking genre expanded

A pivot table can also nest several dimensions on one axis. For example, the analyst can add Year from the Time dimension to the x axis to the pivot table of Figure 2.12. This is shown in Figure 2.14.

Finally, a pivot table can be *pivoted* (as its name suggests!). When pivoting, the dimensions are "rotated." For example, the x and y axes can be swapped, or the Year can be moved from the x axis (as in in Figure 2.14) to the y axis such that there is a row for each genre/year combination.

	Florida		Massachusetts		Virginia		Total
	2008	2009	2008	2009	2008	2009	
Children's books	1013	1029	3784	4312	1452	2579	**14169**
Cooking	8734	12030	9102	10072	240	333	**40511**
Fiction	23458	26687	35497	33447	6872	5621	**131582**
Total	**33205**	**39746**	**48383**	**47831**	**8564**	**8533**	**186262**

Figure 2.14: A pivot table with two dimensions on the x axis

2.8.5 RANKING

When analyzing data, it is often helpful to be able to *rank* it. For example, the book retailer wants to rank the books according to how they sell such that the best selling book is assigned number 1, the second-best selling book is assigned number 2, and so on. It can be relevant to do a complete ranking of all books, but the analyst may also want to find the "top set," e.g., the top-5 best selling books (this is a so-called *top-k* query). The ranking can be based on the number of copies sold, the (aggregated) profit from the books, or on something else. It can also be relevant to do a ranking within another ranking, for example, to find the 5 best selling books within each of the 3 best selling genres.

The MDX query language [82], covered in the next section, offers a number of functions for ranking and sorting multidimensional data.

2.8.6 MULTIDIMENSIONAL QUERYING IN MDX AND SQL

When it comes to the embedding the functionality considered here into a query language, two directions exist: extensions to SQL and dedicated multidimensional languages.

OLAP SQL extensions were pioneered by Gray et al.'s proposal of the CUBE operator [28]. This operator generalizes GROUP BY, crosstabs, and subtotals using the special "ALL" value that denotes that an aggregation has been performed over all values for one or more attributes, thus generating a subtotal or a grand total. "ALL" is thus similar to the $\top$-level in a dimension with a single attribute.

Consider Figure 2.15, which contains a subset of our book retailer's sales data in a standard relational format. The SQL CUBE query below computes the total sales by city and book, with subtotals for both city and book, and a grand total of all sales. The query uses the syntax defined in the SQL:1999 standard.

```
SELECT City, Book, SUM(Sales) AS Sales
FROM SalesTable
GROUP BY CUBE (City, Book)
```

City	Book	Sales
Boston	Gone With the Wind	43765
Miami	Gone With the Wind	21923
Boston	Winnie the Pooh	2345
Miami	Winnie the Pooh	89

Figure 2.15: Sales Table

The query result is shown in Figure 2.16. Note the ALLs that represent the subtotals and the grand total. In most RDBMSs, the ALL value is implemented as a NULL value (possibly with a special function that makes it possible to distinguish it from "real" NULL values).

City	Book	Sales
Boston	Gone With the Wind	43765
Miami	Gone With the Wind	21923
Boston	Winnie the Pooh	2345
Miami	Winnie the Pooh	89
Boston	ALL	46110
Miami	ALL	22012
ALL	Gone With the Wind	65688
ALL	Winnie the Pooh	2434
ALL	ALL	68122

Figure 2.16: Query Result

Another OLAP SQL extension is *window functions* from the SQL:2003 standard. When an aggregate function such as SUM is used in a conventional GROUP BY query, a single result value is produced for a group of rows. With a window function, each row in each implied partition remains, and the aggregate is computed over a set (a "window") of other rows that are related to each row. Consider again the table in Figure 2.15. If we want to compare the amount of copies of each book sold in each city with the total number of sold books in that city, we can use SUM as a window function, as shown in the SQL query here:

```
SELECT City, Book, Sales, SUM(Sales) as Total
OVER (PARTITION BY City)
FROM SalesTable
```

The result of this query is given in Figure 2.17. We partition by City, but the result contains a row for each book sold in each city. The SUM is calculated over all book sales in a city and then associated with each row for a book sale in the city. Thus, all rows for the same city have the same Total value.

City	Book	Sales	Total
Boston	Gone With the Wind	43765	46110
Boston	Winnie the Pooh	2345	46110
Miami	Gone With the Wind	21923	22012
Miami	Winnie the Pooh	89	22012

Figure 2.17: Query result

It is also possible to use RANK that ranks the rows inside each window as a window function. For example, we can rank the books according to sales for each city with the following query.

```
SELECT City, Book, Sales, SUM(Sales) as Total, RANK()
OVER (PARTITION BY City ORDER BY Sales DESC)
FROM SalesTable
```

By now, the SQL standard has adopted the data cube operators [21; 40] along with a range of OLAP functionality such as ranking, percentiles, and windowing, as well as various mathematical and statistical functions. The major RDBMS engines also implement the data cube operators, along with some of the additional functionality mentioned above. However, the syntax may vary slightly from engine to engine. The *iceberg cube* [6] is a further development of the data cube operator that only returns rows where the measure values exceed some threshold (corresponding to interesting "tip of the iceberg" values).

Among the dedicated multidimensional query languages, the by far most prominent and widely used language is MultiDimensional eXpressions (MDX) [82; 89], which originally was proposed by Microsoft, but is now also used in other OLAP products. Unlike the SQL extensions, MDX statements directly produce pivot tables as results, making the integration with OLAP client tools easy.

In MDX, the dimension values are called *members*, and there is a special *Measures* dimension whose values range over the names of the cube measures. This way, any measure value can be referenced using a combination of members (dimension values), including the measure dimension. Dimensions have one or more hierarchies, each with a number of levels. The top level in each dimension is called "(All)" and contains just one one value, like the $\top$ value described earlier. MDX uses dot notation to refer to members, as in `[Time].[Calendar].[Month].[May 2009]`. An MDX query uses a FROM clause for specifying the input cube. A SELECT clause enables the specification of so-called *axes*, the most important axes being the *query axes*. These specify the hierarchies from which to retrieve the data for multiple members. A query can have up to 128 query axes although queries typically have only a few. There are five "standard" axes, called COLUMS, ROWS, PAGES, SECTIONS, and CHAPTERS, that correspond to well-known report concepts. Additionally, a query may specify a *slicer axis* in the WHERE clause that specifies the hierarchies from which to retrieve the data for a single member.

As an example, consider a cube, SalesCube, with two dimensions, City and Book, and one measure, Sales, built on the data in Figure 2.15. Each dimension has just one hierarchy. The City hierarchy contains a (bottom) City level containing the city names and an (All) level containing the single value "All Cities." The Book hierarchy contains a (bottom) Book level containing the book names and an (All) level containing the single value "All Cities." The MDX query below shows the sales per city and book, including subtotals per city and book, and a grand total, i.e., it is the MDX version of the previous SQL CUBE query example on page 25.

```
SELECT
   { [City].Members, [City].[All Cities] } ON COLUMNS,
   { [Book].Members, [Book].[All Books] }  ON ROWS
FROM [SalesCube]
WHERE ( [Measures].[Sales] )
```

Note the use of the "{...}" notation to specify a set of members to show on an axis, the use of the ".Members" notation to specify the set of members in a dimension, and the use of the "[All ...]" members for referring to the top of the dimension hierarchies. The WHERE clause specifies that the Sales measure should be displayed. Figure 2.18 shows the MDX query result.

Book	Boston	Miami	All Cities
Gone With the Wind	43765	21923	65688
Winnie the Pooh	2345	89	2434
All Books	46110	22012	68122

Figure 2.18: MDX query result

MDX also offers a large number of functions that can be used for navigating in hierarchies (some of these "understand" time semantics such that it, e.g., is possible to compare the current month's sales to sales in the same month last year), performing calculations, ranking data, and much more.

2.8.7 GRAPHICAL QUERYING AND VISUALISATIONS

The MultiDimensional eXpressions (MDX) query language is very powerful and allows the analyst to do advanced querying. However, the language is complex and only IT specialists are capable of writing MDX queries. The typical business analyst must then either get help from such a specialist to write and execute the query or must use a more user-friendly system. Therefore, many OLAP clients with rich graphical user interfaces (GUIs) exist. The OLAP clients provide the user with easy-to-use components (e.g., tables and graphs) that can be shown on the GUI. The user can then perform OLAP operations such as drill-down, slicing, or pivoting by clicking or using menus. Behind the scenes, OLAP clients typically generate MDX or SQL queries.

A popular means to give an overview of summarized data such as key performance indicators (KPIs) or total sale compared to last year is a so-called *dashboard* (think also about the dashboard in a car, which should be readable and understandable in the short timeframe the driver has to look at it). A dashboard, thus, shows a limited number of components, or pre-defined analysis results that give a high-level overview. A dashboard may support further analyses of the data, e.g., by enabling drill-down.

2.9 SUMMARIZING EXAMPLE

We proceed to summarize some of the concepts introduced in this chapter by extending the book retailer example. We follow the four-step process from Section 2.7.

In the first step, we choose the business process to model. For a book retailer, an obvious candidate is *sales of books* as this process directly brings profit to the book retailer. Other processes may also be so important that they should be modeled. That may include *inventory* to avoid out-of-stock or over-stock items, which lead to loss of profit, and *customer flow* to optimize staff scheduling so that long customer waiting times are avoided at the cash registers. Here we choose to only consider sales of books.

In the second step, we choose the grain of the business process. At the finest possible grain, we can capture each individual sale of a book. Alternatively, we can capture aggregated data such as total sales per book per store per date. The finest granularity gives the largest flexibility for analysis (the aggregated sales can be found from the non-aggregated sales figures), but more storage is needed and performance may become worse. To see this, assume that our book retailer has 200 shops, each of which on average sells 1000 book copies each day. If we capture each individual sale, we would on average get $200 \cdot 1000 = 200,000$ new facts each day. It is, however, not the case that all books are equally popular. Some books, e.g., the Harry Potter books, sell extremely well, while others are nearly never sold. Assume that 250 different books are sold (in a number of copies—4 on average) from each shop on an average day. If we only capture the aggregated sales per book per shop per date, we would only get $200 \cdot 250 = 50,000$ new facts each day. In other words, by aggregating the sales, we get 75% less facts and thus need much less storage and get better performance, as there is less data to consider in queries. With the aggregated data, there are, however, aspects of the business we cannot investigate. For example, we cannot see during which hours books sell the best, and we cannot use *data mining* to investigate whether certain books are likely to be sold together. It is, thus, very important to understand the needs of the business when creating a multidimensional database. In general, the finest possible granularity should be used. For now, however, we assume that the book retailer only needs the aggregated sales per book per shop per date.

In the third step, we design the dimensions. From the grain "sales per *book* per *shop* per *date*," we can see three (emphasized) dimension candidates. In the examples throughout the chapter, we have used a Location dimension with City as the lowest level. However, the book retailer may have several shops in one city. Thus, we now use a Shop dimension instead. We also rename the Time dimension to the Date dimension to avoid confusion about whether the dimension represents

dates (e.g., May 26th 2010) or specific times (e.g., 11:02 a.m.). When we pick attributes for the dimensions, as much relevant context as possible should be included. The "relevant" context is what business analysts can make use of. For example, it is relevant to know how large a shop is (a large shop should probably sell more than a small shop), but the yearly consumption of electricity in a shop does not seem relevant. Note that there is also a trade-off between the utility of an attribute and the cost or difficulty of obtaining the necessary data. In this example, we choose to include the following attributes.

Book dimension

- BookID (surrogate key)
- ISBN number (a business key as each book has a unique ISBN number)
- Title
- Edition
- Author
- Original title (the book may be a translated publication)
- Publisher
- Published (when the book was released)
- First published (the book may be a republishing)
- Genre
- Language (the book retailer sells books in several languages)
- Number of pages
- Binding (paperback/hard-cover)

Date dimension

- DateID (surrogate key)
- Date
- Month
- Year

Shop dimension

- ShopID (surrogate key)
- Address
- City
- State
- Size (floor space)
- Shelf space
- Opened (the date when the shop opened)
- Has café

The dimension schemas are shown in Figure 2.19. The attributes not shown here are level properties of the lowest level. The hierarchies for the Date and Shop dimensions are straightforward.

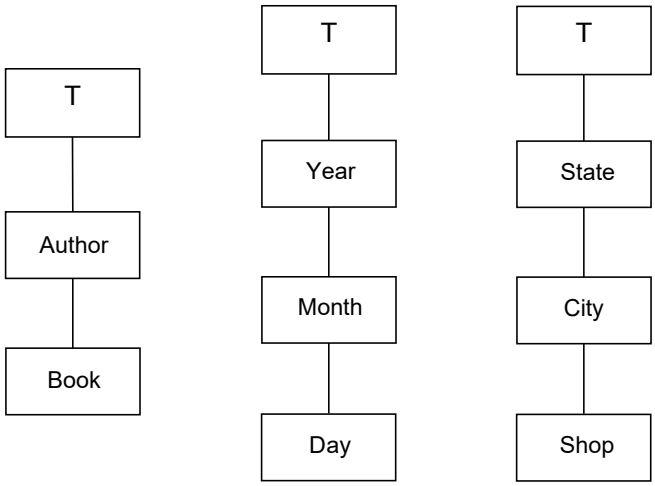


Figure 2.19: Schemas for the dimensions (level properties not shown)

For the Book dimension, we choose to make it possible to roll-up the individual book titles to authors. This will be directly supported by client tools that understand hierarchies (e.g., MDX-based solutions). However, it could be that the analysts need other hierarchies, for example, to roll-up sales based on publisher of the book or genre of the book. We will come back to this in the next chapter.

In the fourth step, we choose measures. An obvious choice is the number of sold books (with the sum formula for aggregation). Also the sales price in dollars for the sold books seems relevant (prices may change over time and location, e.g., due to special campaigns).

This completes the modeling (although a real-world design document would be much more elaborate and precise). We have now modeled a multidimensional database that helps the book retailer's analysts in understanding how books sell. However, our multidimensional model has a number of limitations and inaccuracies. First of all, we only have a single Author attribute. This is fine when a book has a single author. But in the real world, many books are co-authored by several authors. We could then choose to write all their names in the Author attribute or only write the first author's name. However, both of these approaches yield incorrect results when we want to roll-up to the Author level. What we need is a way to represent the many-to-many relationship between books and authors. Regarding the hierarchies, it would also be desirable if we had more hierarchies on the Book dimension. For example, it could make sense to have a hierarchy with the levels Book, Subgenre, Genre, and All, or a hierarchy with the levels Book, Publisher, and All. Another problem is that there is no way to capture changes with the current modeling. A shop may, for example, be expanded such that its floor area increases. If we update the shop's dimension value, old facts will incorrectly be associated with the new size. All of these issues (and many others) are dealt with in the next chapter.

2.10 EXERCISES

1. Illustrate how the data in the cube in Figure 2.20 can be presented in a two-dimensional table by nesting its dimensions. Assume that the cell that cannot be seen in the drawing contains the number 7201.
2. Show how the cube in Figure 2.20 could look if we roll-up to the $\top$ level in the Location dimension.
3. Show how the result from Exercise 2 looks if we drill-down in the Date dimension.
4. Define the schema for a Date dimension with at least 5 levels and identify relevant attributes. Can a Week level be included? Should the attribute Weather be included if the dimension is designed for an ice-cream retailer?
5. For each of the following statements, decide what type (event or state) of fact we are considering.
   (a) A fact exists for (d, p, c, s) if product p was sold to customer c from shop s on day d. The measure is the total price.
   (b) A fact exists for (d, p, s) if product p was sold from shop s on day d. The measure is the total price.
   (c) A fact exists for (d, p, s) for each product p, each shop s, and each day d. The measure is the total inventory of p in s on d.

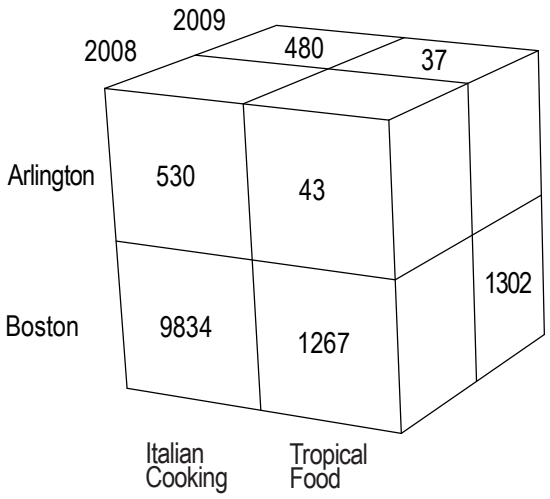


Figure 2.20: A cube

(d) A fact exists for (d, p, s) for each product p, each shop s, and each day d. The measure is the total amount of p sold in s in the year until day d.

(e) A fact exists for (c, e, d) when a call from customer c was answered by employee e on day d. There is no measure.

(f) A fact exists for (c, e, d) when a call from customer c was answered by employee e on day d. The measure is the length of the call in seconds.

6. For each of the following statements, determine whether the measure is additive, semi-additive, or non-additive.

(a) A fact exists for (d, p, c, s) if product p was sold to customer c from shop s on day d. The measure is the total price.

(b) A fact exists for (d, p, s) if product p was sold from shop s on day d. The measure is the total price.

(c) A fact exists for (d, p, s) for each product p, each shop s, and each day d. The measure is the total inventory of p in s on d.

(d) A fact exists for (d, p, s) for each product p, each shop s, and each day d. The measure is the total amount of p sold in s in the year until day d.

(e) A fact exists for (b, c, d) if currency c was exchanged in bank branch b on day d. The measure is the total amount exchanged in US Dollars.

(f) A fact exists for (b, c, d) if currency c was exchanged in bank branch b on day d. The measure is the average exchange rate between c and US Dollars for all transactions involving c on day d.

(g) A fact exists for each (b, c, d) where c is a currency, b a bank branch, and d a date. The measure is the total amount of c kept in b on day d.

7. In this exercise, we consider a chain of stores selling newspapers and magazines. The chain sells many different kinds of publications (e.g., relating to fashion, children, cars, sports) from many different publishers. The types of stores range from small corner shops to super stores with co-located cafés. However, the chain is a bit old-fashioned, as each shop manager at the end of each day has to enter information into a spreadsheet about how many copies were sold of each publication that day. The spreadsheet is then sent to the headquarters. Currently, this is the only way headquarters can collect and analyze sales data from the shops. The management now wishes to gain more insight into the sales of the different publications (and types of publications) from each shop. Design a data warehouse that can help the management. Include relevant hierarchies and attributes. Remember that a certain publication has many "editions."

8. Draw a star schema for the data warehouse designed in Exercise 7.

9. Draw a snowflake schema for the data warehouse designed in Exercise 7.

10. Write SQL queries that find the total sales of sports publications in 2009 when using the star schema and the snowflake schema from the two previous exercises. Discuss pros and cons in each case.